

PISSIR – Plataforma de Jocs Connectats

Projecte Final – Memòria Tècnica

Joc escollit: Breakout (Arkanoid)

**Pau Reverte Garcia
Curs 2025/2026
Alberto Testa**

INDEX

1. Especificacions Funcionals	2
1.1 Descripció del domini d'aplicació	2
1.2 Funcionalitats requerides.....	2
Control Local (Edge)	2
Accessibilitat via Internet (Servidor)	2
1.3 Casos d'ús principals.....	2
2. Anàlisi Tecnològica	3
2.1 Restriccions de xarxa i context	3
2.2 Alternatives analitzades.....	3
3. Elecció de l'Enfocament Tecnològic.....	4
3.1 Motivació de l'elecció REST + MySQL.....	4
3.2 Arquitectura de persistència adoptada.....	4
4. Arquitectura del Programari	5
4.1 Mòduls i funcionalitats	5
4.2 Activitats concurrents (Fils).....	5
5. Descripció de la Implementació.....	6
5.1 Classes implementades.....	6
BreakoutGame (game/BreakoutGame.java)	6
BallThread (game/BallThread.java).....	6
GameController (api/GameController.java)	6
EdgeStorageService (edge/EdgeStorageService.java)	6
SyncService (edge/SyncService.java).....	6
UserService (auth/UserService.java)	6
5.2 Interfície d'usuari	6
6. Validació	7
6.1 Definició dels tests.....	7
6.2 Resultats	7
7. Conclusions	8

1. Especificacions Funcionals

1.1 Descripció del domini d'aplicació

El joc escollit és Breakout, inspirat en l'Arkanoid clàssic. El jugador controla una raqueta horitzontal per fer rebotar una pilota i destruir els blocs situats a la part superior de la pantalla. La forma de guanyar es destruint tots els blocs del nivell sense que caigui la pilota més de 3 vegades. El sistema implementa tres nivells de dificultat (Fàcil, Normal i Difícil) i suporta múltiples instàncies simultànies al mateix ordinador, per poder simular el que podria ser un programa actual a una màquina arcade, per exemple.

La característica principal del projecte és la capacitat Edge (Offline-First): el joc segueix funcionant completament encara que el servidor MySQL de la boira (cloud) no estigui disponible. Quan la connexió es restableix, el SyncService puja automàticament les partides guardades localment.

1.2 Funcionalitats requerides

Control Local (Edge)

- Simulació de la física de la pilota en un fil dedicat (BallThread) a ~20 ticks/segon.
- Gestió de col·lisions: pilota amb blocs, raqueta i parets.
- Mode offline: les partides es desen en un fitxer JSON local (edge_data_<uuid>.json) si el servidor no és accessible.
- Suport multi-instància: cada instància usa un port diferent (port 0 si el 8080 està ocupat).
- Tres nivells de dificultat configurables: Easy, Normal i Hard.
- Power-ups aleatoris, entre x2 de puntuació o una bola extra, que cauen en destruir certs blocs.

Accessibilitat via Internet (Servidor)

- API REST completa per controlar la partida remotament (VisualStudioCode, Postman, etc.).
- Connexió a MySQL per persistir puntuacions i usuaris.
- Sincronització automàtica: SyncService comprova cada 2 segons si hi ha dades locals pendents per pujar.
- Rànquing web: pàgina HTML/JS per consultar les millors puntuacions.
- Sistema de login d'usuaris: registre i autenticació bàsica.

1.3 Casos d'ús principals

- CU1 – Juga en local: el jugador inicia una partida, la pilota es mou en el BallThread i les col·lisions s'actualitzen en temps real.
- CU2 – Finalitza partida: es desa la puntuació a MySQL o al JSON local si no hi ha connexió.
- CU3 – Sincronitza offline: quan MySQL torna, el SyncService detecta el JSON local i puja les partides pendents.
- CU4 – Consulta rànquing: el client accedeix a la pàgina web per veure les millors puntuacions globals.
- CU5 – Control per API: el jugador envia peticions HTTP per moure la raqueta o consultar l'estat de la partida.
- CU6 – Multi-instància: s'obren diverses instàncies del joc al mateix PC sense conflictes de port.

2. Anàlisi Tecnològica

2.1 Restriccions de xarxa i context

El sistema ha de funcionar en xarxes privades sense redireccionament de ports, però s'ha de poder connectar a un servidor MySQL remot per sincronitzar les puntuacions. El tràfic generat és de tipus petició-resposta (REST), amb missatges de mida petita i latència acceptable de fins a 1 segon per a la sincronització de puntuacions.

La principal restricció és la disponibilitat intermitent del servidor: el joc no pot dependre de la connexió constant, d'aquí la necessitat del patró Edge (Offline-First) amb sincronització posterior.

2.2 Alternatives analitzades

Tecnologia	Avantatges	Inconvenients
REST/HTTP (escollit)	Senzill d'implementar, estàndard, ben suportat per Spring Boot	No és temps real, requereix polling per al rànquing
WebSocket	Bidireccional, ideal per notificacions en temps real	Complexitat afegida per a un joc de tornada única
MQTT	Molt lleuger, pub/sub, resistent a offline	Requereix broker dedicat, menys estàndard per a jocs web
gRPC	Baix overhead, tipat fort	Corba d'aprenentatge alta, menys compatible amb navegadors

3. Elecció de l'Enfocament Tecnològic

3.1 Motivació de l'elecció REST + MySQL

He escollit una API REST amb Spring Boot i MySQL com a backend central pels següents motius:

- Simplicitat: REST és el paradigma més conegut i documentat per a aplicacions web, i Spring Boot permet generar un servidor funcional amb molt poc codi de configuració.
- Compatibilitat: la interfície gràfica i la pàgina de rànkning, HTML/JS, poden consumir la mateixa API sense dependències addicionals.
- Escalabilitat horitzontal: si cal, es poden desplegar diverses instàncies del servidor darrere d'un load balancer sense canviar el client.
- Mode offline integrat: gràcies al patró Edge, el client pot funcionar sense el servidor i sincronitzar-se quan la connexió torna.

3.2 Arquitectura de persistència adoptada

S'utilitzen dues capes de persistència complementàries:

- MySQL (online): magatzem principal per a usuaris, partides i rànkning global. S'accedeix via Spring Data JPA.
- JSON local (offline): fitxer `edge_data_<uuid>.json` per cada instància. Cada instància genera un UUID aleatori per evitar col·lisions entre múltiples instàncies al mateix PC.
- SyncService (@Scheduled, cada 2 s): comprova si hi ha dades al JSON local i les puja a MySQL quan la connexió torna.

4. Arquitectura del Programari

4.1 Mòduls i funcionalitats

Paquet / Component	Classes principals	Responsabilitat
game	BreakoutGame, Ball, Paddle, Block, BallThread	Lògica de joc, física de la pilota, col·lisions, power-ups
api	GameController, UserController, RankingController	API REST: endpoints per crear/controlar partides, login i rànkung
persistence	GameRepository, UserRepository (JPA)	Accés a MySQL via Spring Data JPA
edge	EdgeStorageService, SyncService	Persistència local JSON i sincronització automàtica
auth	UserService, UserModel	Registre, login, gestió de sessions (loggedIn)
ui	BreakoutPanel (Swing)	Interfície gràfica clàssica amb Swing

4.2 Activitats concurrents (Fils)

El sistema utilitza el model de concurrència següent:

- BallThread (Runnable): executa el game loop. Separa la simulació física de l'API REST per evitar bloquejos.
- SyncService (@Scheduled): fil secundari que fa ping a MySQL cada 2 segons i puja les partides pendents del JSON local.
- Spring Boot HTTP threads: gestió concurrent de peticions REST per a múltiples instàncies simultànies.
- Gestió de ports: si el port 8080 és ocupat, Spring Boot assigna el port 0 (port lliure aleatori del SO) per suportar multi-instància.

5. Descripció de la Implementació

5.1 Classes implementades

BreakoutGame (game/BreakoutGame.java)

Conté l'estat global d'una partida: llista de blocs, posició de la raqueta i de la pilota, puntuació i nivell de dificultat. Mètodes principals:

- `initGame(difficulty)`: inicialitza coordenades de pilota, raqueta i blocs segons la dificultat.
- `getState()`: retorna l'estat actual serialitzable en JSON per a la API REST.
- `applyPaddleMove(direction)`: actualitza la posició de la raqueta de forma thread-safe (synchronized).

BallThread (game/BallThread.java)

Implementa Runnable. Executa el game loop: mou la pilota, comprova col·lisions i actualitza l'estat. El `Thread.sleep` garanteix que la CPU no es satura i que l'API REST pot respondre mentre el joc corre. Quan detecta fi de partida (pilota perduda o tots els blocs destruïts), notifica el GameController via callback.

GameController (api/GameController.java)

Controlador REST de Spring Boot. Endpoints principals:

- `POST /api/games/start`: crea una partida nova, instancia BallThread i retorna un gameId (hash hexadecimal).
- `GET /api/games/{gameId}/state`: retorna l'estat actual de la partida en JSON.
- `POST /api/games/{gameId}/move`: mou la raqueta (left/right).
- `POST /api/games/{gameId}/end`: finalitza la partida i desa la puntuació.

EdgeStorageService (edge/EdgeStorageService.java)

Gestiona el fitxer JSON local `edge_data_{uuid}.json`. Cada instància del joc genera un UUID únic a l'arrencada per evitar col·lisions entre múltiples instàncies al mateix PC. Proporciona mètodes per desar i llegir partides pendents de sincronitzar.

SyncService (edge/SyncService.java)

Servei Spring amb `@Scheduled (fixedDelay = 2000 ms)`. En cada execució: primer comprova si MySQL és accessible fent un ping lleuger, seguidament si és accessible i hi ha dades al JSON local, les puja a MySQL via GameRepository i buida el fitxer local.

UserService (auth/UserService.java)

Gestiona el registre i el login d'usuaris. El model d'usuari inclou username, imatge de perfil en Base64 i un booleà `loggedIn` per evitar sessions simultànies amb el mateix compte. Les contrasenyes es desen en text pla (decisió pròpia i acadèmica per aconseguir simplicitat; en producció caldria BCrypt).

5.2 Interfície d'usuari

El projecte ofereix dues interfícies complementàries:

- Swing GUI (BreakoutPanel): interfície gràfica clàssica amb visualització en temps real de la pilota, la raqueta i els blocs. Permet control per teclat.
- Rànquing web (HTML/JS): pàgina estàtica que consulta l'endpoint `GET /api/ranking` via `fetch()` i mostra les 10 millors puntuacions.

6. Validació

6.1 Definició dels tests

Els tests estan implementats amb JUnit 5 i Spring Boot Test i cobreixen els escenaris principals del sistema:

ID Test	Escenari	Resultat esperat
T1	Estat inicial de la partida	Pilota i raqueta en posició inicial, puntuació == 0
T2	Col·lisió pilota-paret esquerra	Velocitat x invertida, pilota no surt dels límits
T3	Col·lisió pilota-raqueta	Velocitat y invertida, partida continua
T4	Pilota cau per sota de la raqueta	GameState == FINISHED, puntuació registrada
T5	Destrucció de tots els blocs	GameState == FINISHED (victòria)
T6	Dificultat Easy vs Hard	Velocitats i mides de raqueta correctes per a cada nivell
T7	Persistència offline (sense MySQL)	Partida desada al JSON local sense excepció
T8	Sincronització automàtica	JSON local buidat i dades a MySQL en reconnectar
T9	Multi-instància (+2 jocs simultanis)	Cada instància en port diferent, sense conflictes
T10	Login únic per usuari (loggedIn)	Segon login rebutjat si l'usuari ja té sessió activa
T11	API REST POST /api/games/start	Retorna gameId vàlid i codi HTTP 200
T12	API REST POST /api/games/{id}/move	Raqueta actualitzada correctament al proper tick

6.2 Resultats

Tots els 12 tests passen correctament. En particular:

- T7 i T8 verifiquen el comportament Edge: sense MySQL, les partides es desen al JSON local sense cap excepció visible per l'usuari; quan MySQL torna, el SyncService les puja en menys de 2 segons.
- T9 verifica el suport multi-instància: dues instàncies del joc corren simultàniament al mateix PC sense conflictes de port ni de fitxer JSON (gràcies als UUIDs únics per instància).
- T10 verifica que el sistema de sessions bàsic (loggedIn) impedeix dos logins simultanis amb el mateix compte.

7. Conclusions

El desenvolupament de la plataforma ha permès assolir amb èxit tots els objectius funcionals i tecnològics plantejats inicialment a la guia de la pràctica. L'arquitectura basada en l'enfocament Edge ha demostrat ser una solució altament eficient per garantir la resiliència del sistema, ja que la combinació d'un fitxer JSON local temporal amb el servei de sincronització asíncron protegeix l'experiència de l'usuari davant caigudes del servidor de dades central sense patir cap pèrdua d'informació.

Així mateix, el disseny ha gestionat de manera robusta els reptes de la concurrència gràcies a l'ús d'un fil dedicat exclusivament a les col·lisions de la pilota, fet que desacobla la simulació del rendiment de la xarxa i de l'API REST. Aquesta obertura del joc mitjançant punts d'accés HTTP permet, a més, utilitzar mecanismes de control extern com cURL o Postman.

Finalment, la reconfiguració dinàmica de ports ha solucionat el trepitjat de dades en entorns multi-instància, evidenciant que, tot i els compromisos acadèmics assumits en matèria de seguretat per restriccions de temps, s'ha aconseguit un prototip distribuït completament funcional i interconnectat.

Com a continuació d'aquest projecte, es plantegen les següents millores tecnològiques:

1. Seguretat: Implementar Spring Security i xifrat de contrasenyes mitjançant l'algorisme BCrypt amb tokens JWT per securitzar l'API REST.
2. Migració de dades local: Substituir els fitxers JSON individuals per una base de dades local encastada i thread-safe com SQLite en mode fitxer, millorant el control transaccional offline.
3. Modernització de la Interfície: Evolucionar la interfície Java Swing cap a una arquitectura de client web enriquida connectada per WebSockets per tal de rebre les dades de joc de manera reactiva en temps real.